

Построение пайплайнов данных

Современные хранилища — Lakehouse и паттерны обработки изменений

Дмитрий Малахов



Максим Новосельцев

6 марта 2026 года

Data Lake — обещание и реальность

Традиционный Data Lake (Parquet на S3/HDFS):

	Проблема	Последствия
•  Дешево	✗ Нет ACID-транзакций	Частичные записи, битые данные при сбоях
•  Масштабируемо	✗ Нет версионирования	Нельзя откатиться, нет time travel
•  Любые форматы данных	✗ Конкурентный доступ	Чтение во время записи ломает запросы
	✗ Обновления невозможны	Нужно переписывать целиком

Риски: "Озеро данных" может превратиться в "болото данных" (data swamp)

ACID — зачем это в data lake?

Свойство	Что значит	В data lake
Atomicity	Все или ничего	Нет частичных записей при сбоях
Consistency	Данные валидны	Схема соблюдается
Isolation	Конкурентность не ломает данные	Читаем консистентный снимок
Durability	Записанное не пропадает	Подтверждено — значит сохранено

Почему это сложно на S3/HDFS?

- Объектное хранилище не поддерживает multi-object atomic transactions
- Нет механизма блокировок
- Нужен **дополнительный слой** — табличный формат (table format)

Решение — Open Table Formats

Идея: Прослойка между файлами (Parquet) и движками (Spark, Trino, Flink), которая управляет метаданными, версиями и транзакциями.

Три лидера:

- **Delta Lake** (Databricks → Linux Foundation)
- **Apache Iceberg** (Netflix → Apache)
- **Apache Hudi** (Uber → Apache)

Что они дают: **ACID, time travel, schema evolution, upserts**

Apache Hudi

С самого начала проектировался для инкрементальной обработки и конкурентной записи.

- **Non-Blocking Concurrency Control (NBCC)** — настоящий прорыв
 - В отличие от Optimistic Concurrency Control (OCC), где при конфликте требуется retry
 - NBCC позволяет нескольким потокам писать одновременно без блокировок и откатов
 - Гарантирует сохранение порядка событий (event-time ordering)
- **Индексация записей (Record-level Index)**
 - Мгновенно находит файл с нужной записью без сканирования
 - UPDATE по индексу, а не через метаданные (min/max) как у конкурентов
- **Merge-on-Read** как native-режим (апдейты пишутся в log-файлы, при чтении мержаются)

Delta Lake — глубоко в экосистему Spark

- **Единая архитектура batch + streaming** — один и тот же код для исторической загрузки и реального времени
- **Auto Optimize** — автоматическая оптимизация хранения:
 - Умеет самостоятельно бороться с проблемой маленьких файлов
 - Перед записью распределяет данные так, чтобы файлы получались оптимального размера
 - Пространственная кластеризация для ускорения фильтрации
- **Transaction log** (`_delta_log`) — простой и понятный механизм версионирования
- **UniForm** — возможность читать Delta-таблицы как Iceberg (рядом с метаданными delta могут генериться метаданные iceberg)
- **Ускорение с помощью GPU**

Iceberg — выбор для долгоживущих данных

Особенности:

- Iceberg с самого начала проектировался как спецификация, а не реализация.
- Главная цель — чтобы ваши данные жили дольше, чем любой конкретный движок или облако.
- Один и те же данные читаются из Spark, Trino, Flink (каждый делает то, что умеет лучше).
- Партиционирование не в именах папок, а в метаданных (изменение без перезаписи).
- REST catalog API (единый интерфейс для любого каталога, например Polaris).
- Иерархическая структура метаданных позволяет работать с большим количеством файлов.

Почему стоит использовать по умолчанию:

- Стратегическая цель — избежать vendor lock-in.
- Заранее никогда неизвестно, чем потребуются обрабатывать данные.
- В остальном не сильно хуже остальных инструментов.

Сравнение форматов

Характеристика	Delta Lake	Apache Iceberg	Apache Hudi
ACID-транзакции	✓	✓	✓
Time travel	✓	✓	✓ (CoW only)
Schema evolution	Полная	Полная	Частичная
Real-time ingestion	⚠ Средне	⚠ Средне	✓ Отлично
Мультидвижковость	⚠ Spark в основном	✓ Широкая	✓ Хорошая
Concurrency control	OCC	OCC	NBCC
Partition evolution	Есть	Hidden partitioning	Нет

Как работают транзакции?

Проблема: S3 не поддерживает транзакции на несколько объектов .

Решение — метаданные как точка атомарности:

1. Запись новых data-файлов (новые Parquet).
2. Запись метаданных, указывающих на новые файлы.
3. Атомарное переключение указателя на новые метаданные.

Если упали на шаге 2: новые файлы есть, но никто на них не ссылается — таблица не видит изменений. Транзакция не состоялась.

Если упали на шаге 3: читатели видят старую версию. При повторе — перезаписываем метаданные.

MVCC и Snapshot Isolation

Все три формата используют MVCC (Multi-Version Concurrency Control) :

- Запись А: создает snapshot v1
- Запись Б: начинает, но не мешает чтению v1
- Запись Б: коммитит → snapshot v2
- Чтение: видит либо v1, либо v2 (консистентно)

Snapshot isolation :

- Каждый запрос видит консистентный snapshot таблицы
- Изменения других транзакций не видны до коммита
- Нет "грязного чтения"
- Возможность "путешествовать во времени" и откатываться в случае нахождения ошибок

INSERT, UPDATE, DELETE в одной транзакции

MERGE (UPSERT) — проверяет: если запись есть — обновить, если нет — вставить.

```
MERGE INTO target_table AS target
USING source_table AS source
ON target.key = source.key
WHEN MATCHED THEN UPDATE SET target.col = source.col
WHEN NOT MATCHED THEN INSERT *
```

Что происходит под капотом:

- Находит файлы, содержащие изменяемые записи
- Перезаписывает только эти файлы (не всю таблицу!)
- Добавляет новые файлы для новых записей
- Коммитит транзакцию атомарно

CDC (Change Data Capture)

- Перехватывает INSERT/UPDATE/DELETE в источнике
- Передает их как события с операцией
- Целевая таблица должна применить эти операции

```
OLTP (Postgres, MySQL) → Debezium → Kafka → Stream Processor → Lake
```

Аспект	Delta Lake	Apache Iceberg	Apache Hudi
Streaming engine	Spark Structured Streaming	Flink (best), Spark	Flink, Spark
Нативная поддержка CDC	Через foreachBatch + merge	Через Flink CDC connectors	DeltaStreamer встроенный
Обработка DELETE	Есть (merge)	Есть (merge)	Есть (индекс)
Сложность настройки	Средняя	Средняя	Низкая (готовые утилиты)

Iceberg + Flink (индустриальный стандарт)

```
from pyflink.table import TableEnvironment

t_env.execute_sql("""
    CREATE TABLE cdc_events (
        id INT,
        amount DECIMAL,
        op STRING METADATA FROM 'value.op'
    ) WITH ('connector' = 'kafka', 'format' = 'debezium-json')
""")

t_env.execute_sql("""
    INSERT INTO iceberg_orders
    SELECT id, amount, CURRENT_TIMESTAMP
    FROM cdc_events
""") # Flink сам понимает операции
```

SCD Type 2 : храним историю изменений

Slowly Changing Dimension Type 2:

customer_id	name	city	effective_date	end_date	is_current
1	Иван	Москва	2020-01-01	2023-06-01	false
1	Иван	СПб	2023-06-01	null	true

Iceberg + Flink (real-time SCD):

```
INSERT INTO dim_customer
SELECT
    customer_id,
    name,
    CASE WHEN op = 'u' THEN CURRENT_DATE ELSE effective_date END,
    CASE WHEN op = 'u' THEN NULL ELSE end_date END,
    op != 'u'
FROM cdc_customer;  -- Поток изменений
```

Проблема маленьких файлов (Small File Problem)

- **Почему это плохо:**
 - Метаданные раздуваются
 - List-операции на S3/HDFS медленные
 - Query планировщик захлебывается
 - Чтение тысяч мелких файлов вместо десятков крупных
- **Когда возникает:**
 - Streaming ingestion (каждый микро-батч пишет новый файл)
 - MERGE-операции (создают новые версии файлов)
 - Много мелких апдейтов

Compaction (объединение файлов)

- Для Iceberg нет автоматизации, но она есть для Delta Lake
- Для Iceberg нужно периодически вызывать:

```
# Из Spark
```

```
spark.sql("CALL spark_catalog.system.rewrite_data_files(  
    'prod.db.table', 'sort', 'user_id, date')")
```

```
# Из Trino
```

```
ALTER TABLE iceberg.table EXECUTE OPTIMIZE;
```


Data Skipping и метаданные

- Читаем метаданные (килобайты)
- Смотрим min/max date и amount для каждого файла
- Пропускаем файлы вне диапазона
- Читаем только подходящие файлы (10-20% от всех)

Формат	Где хранится статистика	Что хранится
Delta Lake	Transaction log + checkpoint	min/max, null count
Iceberg	Manifest files	min/max, null count, bloom filter (опц.)
Hudi	.hoodie metadata	min/max, колоночная статистика

Партиционирование — сравнение подходов

Аспект	Delta Lake (традиционное)	Iceberg (hidden)	Hudi
Где хранится	В пути к файлу	В метаданных	В метаданных + путь
Изменение схемы	Требует перезаписи	Можно без перезаписи	Ограничено
Эволюция	Сложная	Полная	Частичная

Snapshot Management (Iceberg)

-- Посмотреть историю

```
SELECT * FROM prod.db.table.history;
```

-- Time travel

```
SELECT * FROM prod.db.table FOR SYSTEM_VERSION AS OF 12345;
```

```
SELECT * FROM prod.db.table FOR SYSTEM_TIME AS OF '2024-01-01';
```

-- Очистка

```
CALL spark_catalog.system.expire_snapshots('prod.db.table', TIMESTAMP '2024-01-01');
```

Lakehouse — объединение озера и склада

Lakehouse — это архитектурный паттерн, который объединяет:

От Data Lake

✓ Дешевое объектное хранилище (S3, ADLS, GCS)

✓ Масштабирование до petabytes

✓ Любые форматы данных

✓ Единое хранилище для всех данных

От Data Warehouse

✓ ACID-транзакции

✓ Версионирование (time travel)

✓ Schema enforcement

✓ Оптимизации запросов

Домашнее задание

1. Провести эксперименты с одновременной записью в рассмотренные форматы
 1. С какими проблемами можно столкнуться?
 2. Кто-как эти проблемы обрабатывает?
2. Провести тест производительности рассмотренных форматов:
 1. Кто меньше весит?
 2. Кого быстрее читать?
 3. Кого быстрее писать (в разных объемах: 10%, 20%, 50% или 100%)?
3. Привести кейс, когда оправдано использовать Hudi или Delta Lake вместо Iceberg.

